Questions:
Here are the relevant questions extracted from the text:

1. Choose an example for JAVA IDE: A.Notepad B.JAVAC C.JVM D.Netbeans
2. List any two types of constructors in JAVA.
3. Which of the following is not an OOPs concept in Java? A.Inheritance B.Encapsulation C.Polymorphism D.Compilation
4. Show the method to restrict a member of a class from inheriting to its subclasses.
5. Write the steps to create a package in JAVA.
6. Tell the main uses of the super keyword?

Answers:

Question 1: Here are the relevant questions extracted from the text:

This module provides a foundational understanding of Object-Oriented Programming (OOP) and an introduction to Java.

Summary of the Module

This module introduces Object-Oriented Programming (OOP) as a paradigm focused on data and its associated functions (methods), protecting data from unintentional modification. It contrasts OOP with the procedural approach, which emphasizes program logic.

Key OOP Concepts:

• **Objects:** The basic entities in OOP, representing real-world items with properties (data/attributes) and behaviors (methods). An object is a combination of data and methods.
• **Classes:** Logical abstractions that describe the form and behavior of an object. A class is a template or blueprint for creating objects, much like a `data type` is to a `variable`. Objects are instances of classes. Class members include instance variables (data) and member methods (code that operates on the data).

Major Characteristics (Concepts/Principles) of OOP:

1. **Abstraction:** Hiding implementation details and showing only essential functionality to the user. Focuses on "what" an object does, not "how."
2. **Encapsulation:** Wrapping data and methods into a single unit (a class). It controls access to data, protecting it from external interference.
3. **Inheritance:** A mechanism for one class to acquire properties and behaviors from another class, promoting reusability and hierarchical classification. The new class inherits general attributes from its parent.
4. **Polymorphism:** The ability to perform one task in different ways based on context. Method overloading in Java is a prime example, allowing methods with the same name but different parameters.

Features/Advantages of Java:

Java is highlighted as a simple, pure object-oriented, platform-independent (or platform-neutral), compiled and interpreted, robust, secure, multithreaded, architectural-neutral, high-performance, and dynamic language. Its main advantages are **Portability, Security, and Reusability**.

Java Programming Tools:

• **JDK (Java Development Kit):** Essential for compiling and running Java programs. It includes:
• `javac`: The Java compiler, which converts Java source code into bytecode.
• `java`: Used to execute the bytecode.
• **IDEs (Integrated Development Environments):** User-friendly tools like NetBeans and Eclipse that simplify editing, compiling, and executing Java programs.

Java Virtual Machine (JVM):

The JVM is a runtime environment that interprets and executes the bytecode generated by the Java compiler. It's crucial for Java's portability, enabling the same Java program to run on various platforms.

Defining Objects and Classes:

A class is defined using the `class` keyword and contains instance variables (properties/attributes) and methods (behaviors/actions) that operate on those variables. Example: A `Vehicle` class might have `passengers`, `fuelCap`, `mpg` as instance variables and `range()` as a method.

Short Quiz (5 Questions)

1. **Question:** In Object-Oriented Programming (OOP), what is the primary difference in focus compared to the procedural approach (like C language)?
• **Answer:** OOP primarily focuses on data and its associated functions (methods) and protecting that data, whereas the procedural approach gives importance to program logic on how to solve a problem.

2. **Question:** Explain the relationship between a "class" and an "object" in OOP using a simple analogy from the text.
• **Answer:** A class is a logical abstraction, template, or blueprint for an object, similar to how a 'data type' is to a 'variable'. An object is an instance of that class. For example, 'fruit' is a class, and 'orange', 'mango', 'apples' are objects of the fruit class.

3. **Question:** Which of the four major OOP characteristics is described as "the process of hiding the implementation details and shows only the functionality to the user"?
• **Answer:** Abstraction.

4. **Question:** What are the two main programs included in the JDK (Java Development Kit), and what is the specific role of each?
• **Answer:** The two main programs are `javac` and `java`. `javac` is the Java compiler used to generate bytecode from Java source programs, and `java` is used to run the compiled Java program (bytecode).

5. **Question:** How does the Java Virtual Machine (JVM) contribute to Java's key advantage of "portability"?
• **Answer:** The JVM provides a runtime environment that interprets the bytecode generated by the Java compiler. Because the bytecode is platform-independent, the JVM acts as a translator for the specific

operating system, allowing the same Java program (bytecode) to be executed on multiple platforms without modification.

Question 3: 1. Choose an example for JAVA IDE: A.Notepad B.JAVAC C.JVM D.Netbeans

Here's a summary of the provided text and some quiz questions based on it.

Summary of MODULE-1: OBJECT ORIENTED PROGRAMMING

This module introduces Object-Oriented Programming (OOP), emphasizing an approach where data and its associated functions (methods) are given primary importance, protecting data from unintentional modification.

Core OOP Concepts:
• **Objects:** Fundamental entities that combine properties (data/attributes) and behaviors (methods). For example, an `Employee` object has `name`, `department` (data) and `taxCalculation` (method).
• **Classes:** Logical abstractions, templates, or blueprints for creating objects. A class defines the form and behavior, acting like a 'data type' while objects are 'variables' of that type. A class consists of **member variables** (data fields) and **member methods** (functions that operate on the data).

Major Characteristics (Pillars) of OOP:
1. **Abstraction:** Hiding implementation details and exposing only essential functionality to the user. Focuses on "what" an object does, not "how."
2. **Encapsulation:** Binding data and methods together into a single unit (a class). It controls access to data, protecting it from external misuse.
3. **Inheritance:** A mechanism for one class (subclass) to acquire properties and behaviors of another class (superclass). It promotes reusability, allowing new features to be added without modifying existing code.
4. **Polymorphism:** The ability for an entity to take on multiple forms, meaning a single task can be performed in different ways based on context. In Java, method overloading (same method name, different parameters) is an example.

Features and Advantages of Java:
Java is highlighted as a simple, pure object-oriented, platform-independent, compiled and interpreted, robust, secure, multithreaded, architectural neutral, high-performance, and dynamic language. Its main advantages are **Portability, Security, and Reusability**.

Java Programming Tools:
• **JDK (Java Development Kit):** Required to compile and run Java programs.
• `javac`: The Java compiler, which converts source code (`.java` files) into bytecode (`.class` files).
• `java`: The Java application launcher, which runs the bytecode.
• **IDEs (Integrated Development Environments):** User-friendly tools like NetBeans and Eclipse that facilitate editing, compiling, and executing Java programs.

Java Virtual Machine (JVM):
A program that provides a runtime environment for Java. It interprets the bytecode generated by `javac` and executes the code. The JVM is crucial for Java's **portability**, as the same bytecode can run on any platform with a compatible JVM.

Defining Objects and Classes:
Classes are defined using the `class` keyword and contain **instance variables** (attributes like `radius`, `length`, `passengers`) and **methods** (behaviors like `getArea()`, `range()`).

Quiz Questions

Multiple Choice Questions:

1. Which of the following is NOT a major characteristic (principle) of OOP?
a) Abstraction
b) Debugging
c) Inheritance
d) Polymorphism

2. In OOP, what is an object fundamentally a combination of?
a) Only data
b) Only methods
c) Data and methods
d) Program logic and memory

3. What is a class best described as in OOP?
a) A variable of a specific type
b) An instance of an object
c) A template or blueprint for objects
d) A function that operates on data

4. Which OOP principle involves hiding the implementation details and showing only the functionality to the user?
a) Encapsulation
b) Inheritance
c) Abstraction
d) Polymorphism

5. Which Java tool is responsible for converting Java source code into bytecode?
a) `java`
b) `javac`
c) JVM
d) IDE

6. The portability of a Java program is primarily achieved by:
a) Its multithreaded nature
b) The use of IDEs like Eclipse
c) Bytecode and JVM mechanism

d) Strong memory management

7. Which of the following is an example of Polymorphism in Java, as mentioned in the text?
a) Wrapping data and methods into a single unit
b) An object acquiring properties of another class
c) Hiding implementation details
d) Method overloading

8. The main advantages of Java, as stated in the text, are:
a) Speed, Simplicity, Security
b) Portability, Security, Reusability
c) Performance, Multithreading, Debugging
d) Object-Oriented, Dynamic, Easy to Learn

Short Answer Questions:

9. Explain the difference between an "object" and a "class" in OOP, using the analogy provided in the text.
10. What is Encapsulation, and what is considered the basic unit of encapsulation in Java?
11. List three features or advantages of Java mentioned in the text (besides the main advantages).
12. What is the role of the Java Virtual Machine (JVM)?
13. If you want to run an already compiled Java program from the command line, which JDK program would you use?
14. How does inheritance support the idea of reusability?

Answer Key:

1. b) Debugging
2. c) Data and methods
3. c) A template or blueprint for objects
4. c) Abstraction
5. b) `javac`
6. c) Bytecode and JVM mechanism
7. d) Method overloading
8. b) Portability, Security, Reusability

9. A **class** is a logical abstraction, a template, or a blueprint (like a 'data type'). An **object** is an instance of that class, a concrete entity created from the blueprint (like a 'variable' of that type).
10. Encapsulation is the process of wrapping (binding) the data and methods together into a single unit. In Java, a **class** is the basic unit of encapsulation.
11. Any three from: Simple and easy to learn, pure object-oriented language, platform independent, compiled and interpreted language, robust and secure, multithreaded, architectural neutral, high performance, dynamic language.
12. JVM is a program that provides a runtime environment, interprets the bytecode generated by the Java compiler, and executes the code.
13. `java`
14. Inheritance provides reusability by allowing a new class to acquire properties and behaviors from an existing class (its parent). This means you can add additional features to an existing class without modifying it, and the new class will have the combined features of both, reusing the parent's code.

Question 4: 2. List any two types of constructors in JAVA.

Here's a summary of the provided text and five multiple-choice questions based on it:

Summary of Module-1: Object-Oriented Programming

This module introduces Object-Oriented Programming (OOP) as an approach that prioritizes data and its associated functions (methods), protecting data from unintended modification. Unlike procedural programming (e.g., C), which focuses on program logic, OOP uses **objects** as fundamental entities, each possessing **properties (data)** and **behaviors (methods)**. An object is thus a combination of data and methods.

A **class** is defined as a logical abstraction, a template, or a blueprint that describes the form and behavior of an object. An object is an **instance** of a class. Analogously, a class is like a 'data type' and an object is a 'variable' of that type. To create objects, one must first define a class. The data within a class are called **member variables** or **instance variables**, and the code that operates on them are **member methods** or just **methods**.

The four major characteristics (concepts/principles) of OOP are:
1. **Abstraction:** Hiding implementation details and showing only essential functionality to the user, focusing on "what" an object does rather than "how."
2. **Encapsulation:** Binding (wrapping) data and methods together into a single unit (a class), controlling access to the data from outside the class. A class is the basic unit of encapsulation in Java.
3. **Inheritance:** A mechanism for one class to acquire properties and behaviors from another class, promoting hierarchical classification and **reusability**. It allows adding features to an existing class without modifying it.
4. **Polymorphism:** The ability to perform one task in different ways based on context. In Java, method overloading (methods with the same name but different parameters) is an example.

Java is highlighted for its key features and advantages:
• **Simple, Pure OOP, Platform Independent, Compiled & Interpreted, Robust & Secure** (due to strong memory management and no pointers), **Multithreaded, Architectural Neutral, High Performance, and Dynamic.**
• Its main advantages are **Portability, Security, and Reusability**.

Java Programming Tools include the **JDK (Java Development Kit)**, which contains `javac` (the Java compiler for generating bytecode) and `java` (used to run programs). Integrated Development Environments (IDEs) like NetBeans and Eclipse offer user-friendly interfaces.

The **Java Virtual Machine (JVM)** is crucial for Java's portability. It's a runtime environment that interprets and executes the bytecode generated by the Java compiler, allowing the same Java program to run on multiple platforms.

Finally, the text outlines how to define a class using the `class` keyword, specifying instance variables (data fields) and methods. It uses a `Vehicle` class example with `passengers`, `fuleCap`, `mpg` as instance variables and `range()` as a method to illustrate these concepts.

Multiple-Choice Questions

1. What is the primary focus of Object-Oriented Programming (OOP) as described in the text?
a) Program logic and sequential execution.
b) Data and its associated functions (methods).
c) Hardware-level memory management.
d) Optimizing compilation speed.

2. Which OOP principle involves hiding the implementation details and showing only the functionality to the user?
a) Encapsulation
b) Inheritance
c) Abstraction
d) Polymorphism

3. How does Java primarily achieve its "platform independence" or "portability"?
a) By using a special operating system.
b) Through the `javac` compiler directly running code.
c) Via bytecode interpreted by the Java Virtual Machine (JVM).
d) By not requiring any compilation step.

4. In OOP, what is an "object" in relation to a "class"?
a) An object is a template for a class.
b) An object is a logical abstraction describing a class.
c) An object is an instance of a class.
d) An object is a collection of similar classes.

5. Which component of the Java Development Kit (JDK) is responsible for compiling Java source programs into bytecode?
a) `java`
b) JVM
c) IDE (e.g., NetBeans)
d) `javac`

Answer Key:
1. b) Data and its associated functions (methods).
2. c) Abstraction
3. c) Via bytecode interpreted by the Java Virtual Machine (JVM).
4. c) An object is an instance of a class.
5. d) `javac`

Question 5: 3. Which of the following is not an OOPs concept in Java? A.Inheritance B.Encapsulation C.Polymorphism D.Compilation

This module provides a comprehensive introduction to Object-Oriented Programming (OOP) and its core concepts, specifically within the context of Java.

Summary of the Provided Text

The provided text introduces **Object-Oriented Programming (OOP)** as an approach prioritizing data and its associated functions (methods), protecting data from unintentional modification. It contrasts with procedural programming (like C), which focuses on program logic. In OOP, **objects** are fundamental entities combining properties (data/instance variables) and behaviors (methods). A **class** acts as a logical blueprint or template for objects, defining their form and behavior; an object is an **instance** of a class. The text likens a class to a 'data type' and an object to a 'variable' of that type.

The four major characteristics (concepts/principles) of OOP are detailed:
1. **Abstraction:** Hiding implementation details and exposing only essential functionality to the user, focusing on "what" an object does rather than "how."
2. **Encapsulation:** The process of bundling data and methods into a single unit (a class in Java), controlling access to the data.
3. **Inheritance:** Allows objects of one class to acquire properties of another, supporting hierarchical classification and promoting **reusability** by enabling the addition of new features without modifying existing code.
4. **Polymorphism:** The ability to perform one task in multiple ways based on context. Method overloading in Java is cited as an example.

The text then highlights **Java's features and advantages**: it is simple, pure object-oriented, platform-independent (or neutral), compiled and interpreted, robust, secure (strong memory management, no pointers), multithreaded, architectural neutral, high-performance, and dynamic. The main advantages are **Portability, Security, and Reusability**.

Java Programming Tools include the **JDK (Java Development Kit)**, which provides `javac` (the Java compiler for generating bytecode) and `java` (to run programs). Integrated Development Environments (IDEs) like NetBeans and Eclipse offer user-friendly interfaces for development.

The **Java Virtual Machine (JVM)** is explained as a runtime environment that interprets and executes the bytecode generated by `javac`. This bytecode and JVM mechanism are crucial for achieving Java's **portability**.

Finally, the text delves into **Objects and Classes**, describing how to define a class using the `class` keyword. A class contains instance variables (data fields representing properties/attributes like `radius` for a `Circle` or `passengers` for a `Vehicle`) and methods (actions/behaviors like `getArea()` or `range()`) that operate on these variables.

---

10 Questions from the Text

1. What is the fundamental difference in focus between Object-Oriented Programming (OOP) and the procedural approach to programming?
2. Define 'object' and 'class' in the context of OOP, and explain their relationship using an analogy provided in the text.
3. Explain the concept of **Abstraction** in OOP. What aspect does it focus on, and what does it hide?
4. Describe **Encapsulation**. What is considered the basic unit of encapsulation in Java, and what is its primary purpose?
5. How does **Inheritance** promote the concept of "reusability" in OOP?
6. What is **Polymorphism**, and which Java feature is given as an example of it in the text?
7. List three main advantages of Java programming mentioned in the text.
8. Identify the two primary command-line programs included in the JDK and state their respective functions.
9. Explain the role of the **Java Virtual Machine (JVM)** in achieving the "platform independence" of Java programs.
10. Describe the general structure of a class definition in Java, specifying what 'instance variables' and 'methods' represent within that class.

Question 6: 4. Show the method to restrict a member of a class from inheriting to its subclasses.

Here's a summary of the provided text and 5 multiple-choice questions based on it:

Summary of Module-1: Object-Oriented Programming

This module introduces Object-Oriented Programming (OOP) as an approach that prioritizes data and its associated functions (methods), protecting data from unintended changes. Unlike procedural programming which focuses on logic, OOP builds programs around **objects**, which are basic entities combining properties (data/attributes) and behaviors (methods).

A **class** is a logical blueprint or template that defines the structure and behavior for objects. An **object** is an instance of a class, akin to a variable of a specific data type. Classes contain **member variables** (instance variables) for data and **member methods** for operations.

The four major characteristics (principles) of OOP are:
1. **Abstraction:** Hiding implementation details and showing only essential functionality.
2. **Encapsulation:** Binding data and methods into a single unit (a class), controlling data access.
3. **Inheritance:** Allowing a class to acquire properties of another, promoting reusability and hierarchical classification.
4. **Polymorphism:** Enabling one task to be performed in different ways, often through methods with the same name but different parameters (method overloading).

Java is highlighted as a simple, pure object-oriented, platform-independent, compiled and interpreted, robust, secure, multithreaded, architectural neutral, high-performance, and dynamic language. Its main advantages are portability, security, and reusability.

To work with Java, the **JDK (Java Development Kit)** is required, which includes `javac` (the compiler for generating bytecode) and `java` (the interpreter for running bytecode). IDEs like NetBeans and Eclipse offer user-friendly development environments.

The **Java Virtual Machine (JVM)** is crucial for Java's portability. It provides a runtime environment and interprets the platform-independent bytecode, allowing the same Java program to run on various operating systems.

Finally, the module explains how to **define a class** using the `class` keyword. A class encapsulates instance variables (attributes like `radius`, `length`) and methods (behaviors like `getArea()`, `range()`) that operate on that data.

Multiple Choice Questions:

1. **Which of the following best describes an object in Object-Oriented Programming (OOP)?**
a) A program logic that solves a problem.
b) A combination of data (properties) and methods (behavior).
c) A template or blueprint for creating classes.
d) A function that protects data from modification.

Correct Answer: b) A combination of data (properties) and methods (behavior).

2. **The OOP characteristic that involves hiding the implementation details and showing only the functionality to the user is called:**
a) Encapsulation
b) Inheritance
c) Abstraction
d) Polymorphism

Correct Answer: c) Abstraction

3. **How does Java achieve its platform independence (portability)?**
a) By using direct machine code compilation for each platform.
b) By relying on the operating system to interpret source code.
c) Through the generation of bytecode which is interpreted by the Java Virtual Machine (JVM).
d) By strictly avoiding the use of any external libraries.

Correct Answer: c) Through the generation of bytecode which is interpreted by the Java Virtual Machine (JVM).

4. **In Java, which JDK tool is responsible for compiling Java source programs (`.java` files) into bytecode (`.class` files)?**
a) `java`
b) `javac`
c) NetBeans

d) JVM

Correct Answer: b) `javac`

5. **Which OOP principle primarily promotes the idea of reusability, allowing new classes to acquire properties and methods from existing ones without modification?**
a) Abstraction
b) Encapsulation
c) Polymorphism
d) Inheritance

Correct Answer: d) Inheritance

Question 7: 5. Write the steps to create a package in JAVA.

Here's a summary of the provided text and 5 multiple-choice questions based on it:

Summary of Module-1: Object Oriented Programming

This module introduces Object-Oriented Programming (OOP) as an approach that prioritizes data and its associated functions (methods) to protect data from accidental modification. Unlike procedural programming (e.g., C), OOP focuses on "objects," which are basic entities combining properties (data/attributes) and behaviors (methods).

A **class** is a logical blueprint or template that defines the form and behavior for objects. An **object** is an instance or variable of a class. The data within a class are called **member variables** or **instance variables**, and the code operating on them are **member methods** or **methods**.

The four major characteristics (principles) of OOP are:
1. **Abstraction:** Hiding implementation details, showing only essential functionality ("what" not "how").
2. **Encapsulation:** Binding data and methods into a single unit (a class), controlling access to data.
3. **Inheritance:** A mechanism for one class to acquire properties and behaviors of another, promoting reusability and hierarchical classification.
4. **Polymorphism:** The ability to perform a task in different ways based on context, exemplified by method overloading in Java (same method name, different parameters).

Java is highlighted as a pure object-oriented language with several advantages: simplicity, platform independence (achieved via bytecode and JVM), robustness, security, multithreading, architectural neutrality, high performance, and dynamism. Its main advantages are Portability, Security, and Reusability.

To develop Java programs, the **JDK (Java Development Kit)** is required, which includes `javac` (the Java compiler) and `java` (the program launcher). **IDEs** like NetBeans and Eclipse offer user-friendly development environments.

The **Java Virtual Machine (JVM)** is crucial for Java's portability; it's a runtime environment that interprets the `javac`-generated bytecode and executes the program across various platforms.

Finally, the module explains how to define a class using the `class` keyword. A class encapsulates instance variables (data fields like `radius`, `length`) representing an object's state, and methods (like `getArea()`, `range()`) representing its behavior.

Multiple Choice Questions (MCQs)

1. What is the primary focus of Object-Oriented Programming (OOP) as described in the text?
a) Program logic and sequential execution.
b) Data and its associated functions (methods).
c) Low-level memory management using pointers.
d) Optimizing compilation speed.

2. Which OOP principle is described as the process of hiding implementation details and showing only the functionality to the user?
a) Encapsulation
b) Inheritance
c) Polymorphism
d) Abstraction

3. According to the text, which of the following is *not* explicitly listed as one of the *main advantages* of Java?
a) Portability
b) Security
c) High Performance
d) Reusability

4. What is the role of `javac` in the Java programming toolkit?
a) To run the Java program on the JVM.
b) To interpret bytecode at runtime.
c) To compile Java source programs into bytecode.
d) To provide a user-friendly graphical interface for coding.

5. How does the Java Virtual Machine (JVM) contribute to Java's portability?
a) By compiling Java source code directly into machine-specific executables.
b) By allowing Java programs to be written without any classes or objects.
c) By interpreting the bytecode generated by the Java compiler, enabling execution on multiple platforms.
d) By providing direct access to system hardware, bypassing the operating system.

Answer Key:
1. b) Data and its associated functions (methods).
2. d) Abstraction
3. c) High Performance (The text lists it as a feature, but explicitly states the *main advantages* are Portability, Security, and Reusability.)

4. c) To compile Java source programs into bytecode.
5. c) By interpreting the bytecode generated by the Java compiler, enabling execution on multiple platforms.

Question 8: 6. Tell the main uses of the super keyword?

Here's a summary of the provided text, key term explanations, and a short quiz.

Summary of MODULE-1: OBJECT ORIENTED PROGRAMMING

This module introduces Object-Oriented Programming (OOP) as a paradigm that prioritizes data and its associated functions (methods) to protect data from unintended modifications. Unlike procedural programming which focuses on logic, OOP builds programs around **objects**, which are basic entities representing real-world items. Each object possesses **properties (data)** and **behaviors (methods)**.

A **class** is a logical blueprint or template that defines the form and behavior for objects. An object is an **instance** of a class. Classes contain **member variables** (or instance variables) to represent data, and **member methods** to operate on that data.

The four major characteristics (principles) of OOP are:
1. **Abstraction:** Hiding implementation details and showing only necessary functionality.
2. **Encapsulation:** Binding data and methods into a single unit (a class), controlling access to data.
3. **Inheritance:** Allowing one class to acquire properties of another, promoting reusability and hierarchical classification.
4. **Polymorphism:** Performing a single task in multiple ways, such as method overloading in Java.

Java is highlighted as a pure object-oriented, simple, platform-independent, robust, secure, multithreaded, and dynamic language. Its key advantages are **Portability, Security, and Reusability**.

To develop Java programs, the **JDK (Java Development Kit)** is required, containing `javac` (compiler to bytecode) and `java` (program runner). IDEs like NetBeans and Eclipse also aid development. The **Java Virtual Machine (JVM)** is crucial for Java's portability; it interprets the compiled `bytecode` and executes it on any platform, allowing the same Java program to run everywhere.

Defining a class involves using the `class` keyword, followed by instance variables (attributes) and methods (behaviors).

Key Term Explanations

Here are explanations for the key terms found in the text:

1. **Object:** A basic entity in OOP that combines data (properties) and functions (methods) into a single unit. It represents a real-world entity or data item.
2. **Method:** A function associated with an object's data, defining its behavior.
3. **Class:** A logical blueprint or template that describes the form (data) and behavior (methods) of a set of objects. Objects are instances of a class.
4. **Member Variables / Instance Variables:** Data fields defined within a class that represent the properties or attributes of an object.
5. **Member Methods:** Code (functions) defined within a class that operate on the object's data and define its behavior.
6. **Abstraction:** An OOP principle that involves hiding the implementation details and showing only the essential functionality to the user, focusing on "what" an object does rather than "how."
7. **Encapsulation:** An OOP principle that involves wrapping (binding) data and methods together into a single unit (a class) and controlling access to that data.
8. **Inheritance:** An OOP principle where an object of one class can acquire the properties and behaviors of objects from another class, promoting reusability and hierarchical relationships.
9. **Polymorphism:** An OOP principle that allows a single task to be performed in different ways based on the context. In Java, method overloading is an example.
10. **JDK (Java Development Kit):** A software development environment used for developing Java applications. It includes tools like the Java compiler (`javac`) and the Java runtime tool (`java`).
11. **JVM (Java Virtual Machine):** A program that provides a runtime environment for Java programs. It interprets the bytecode generated by the Java compiler and executes the code, enabling Java's platform independence.
12. **Bytecode:** The intermediate machine-independent code generated by the Java compiler (`javac`) from Java source code. This bytecode is then interpreted and executed by the JVM.

Quiz

Instructions: Choose the best answer or fill in the blank.

1. Which of the following is NOT a major characteristic of Object-Oriented Programming?
a) Abstraction
b) Encapsulation
c) Compilation
d) Inheritance

2. A _____ is a logical blueprint or template for objects, while an _____ is an instance of that blueprint.

3. The OOP principle of hiding implementation details and showing only essential functionality is known as:
a) Encapsulation
b) Polymorphism
c) Abstraction
d) Inheritance

4. What Java tool is responsible for compiling Java source code into bytecode?
a) `java`
b) `javac`
c) JVM

d) JDK

5. True or False: Java achieves platform independence because its bytecode can be executed on any system by the Java Virtual Machine (JVM).

6. Which OOP principle allows a method to have the same name but different parameters, as exemplified by method overloading in Java?
a) Inheritance
b) Abstraction
c) Encapsulation
d) Polymorphism

Quiz Answers:

1. c) Compilation
2. Class, Object
3. c) Abstraction
4. b) `javac`
5. True
6. d) Polymorphism
